Web Recommender Systems 2025: Project

Davide Marchi
lnx547@alumni.ku.dk

1 Week 6 [Feb 3 - Feb 9]: Familiarize Yourself with the Datasets

This week focused on understanding the dataset and preparing it for recommendation tasks. The goal was to clean the data, explore its properties, and extract key statistics to gain insights into user-item interactions. This preprocessing step was essential to ensure data quality for later modeling.

1.1 Dataset Description

The dataset used in this project is the "Musical Instruments" dataset from the Amazon Review data 2023 (5-core subset). The dataset consists of 14,000 item ratings from nearly 900 users on approximately 500 items. The data is split into training and test sets, with 80% of the data allocated for training ("train.tsv") and 20% for testing ("test.tsv").

1.2 Data Cleaning and Preprocessing

To ensure the quality of the dataset, I performed the following preprocessing steps:

1. Removal of missing values (entries with missing user ID, item ID, or rating).
   Outcome: 0 ratings removed.

2. Deduplication by keeping the most recent rating when a user rated the same item multiple times.
   Outcome: 1087 ratings removed from the training set and 1178 from test set.

3. Ensuring that all users in the test set are also present in the training set.
   Outcome: 0 users removed.

In terms of dimensions the effect of the cleaning can be summarized in Table 1 (for the ratings) and Table 2 (for the users).

Dataset	Before Cleaning	After Cleaning
Train Set	11000	9913
Test Set	3000	1822

Table 1: Effect of the Data Cleaning Process on the Number of Ratings

Dataset	Before Cleaning	After Cleaning
Train Set	800	800
Test Set	437	437

Table 2: Effect of the Data Cleaning Process on the Number of Users

1.3 Exploratory Data Analysis and Statistics

To gain insights into the dataset, I computed the following statistics on the training set:

- Distribution of ratings per user in Figure 1
- Distribution of ratings per item in Figure 2
- Overall rating value distribution in Figure 3

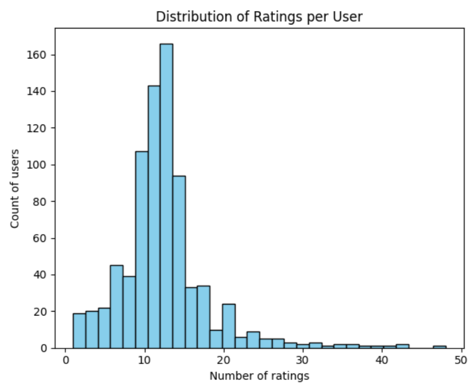

Figure 1: Distribution of Ratings per User

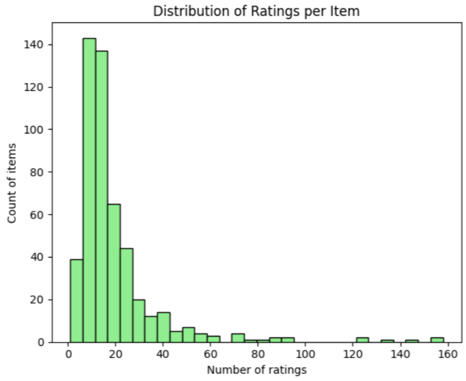

Figure 2: Distribution of Ratings per Item

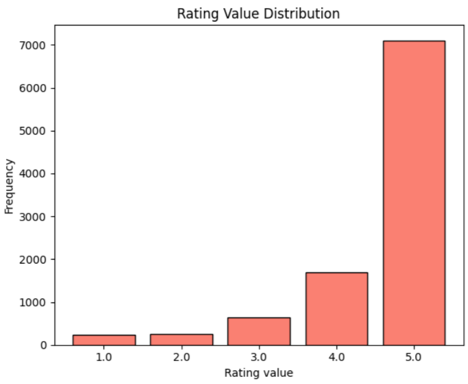

Figure 3: Distribution of Ratings per Value

Also, to show the long-tail distribution of ratings, I plotted the number of ratings per item in Figure 4, computed on the training set.

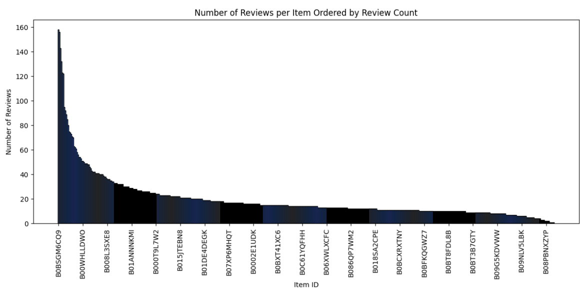

Figure 4: Long tail distribution of rating over all the items

1.4 Highly Rated Items

To identify popular items, I computed the number of ratings ≥ 3 for each item and reported the top 5 most highly rated items in Table 3.

Item ID	High Ratings	Average Rating
B0BSGM6CQ9	153	4.6899
B0BPJ4Q6FJ	153	4.7372
B09857JRP2	141	4.7692
B0BCK6L7S5	120	4.3030
B0BTC9YJ2W	119	4.7377

Table 3: Top 5 Most Highly Rated Items

1.5 Discussion

The dataset exhibits common characteristics of user-item interactions:

- There is a long-tail distribution where a small number of items receive most of the ratings, while many items have very few ratings.
- The majority of users have rated only a very small percentage of the available items, which can lead to sparsity issues in collaborative filtering approaches. Indeed, the user-item matrix will be mostly empty, so there are often very few overlapping ratings between users. This makes it harder for algorithms like kNN to find similar users, and for models like SVD to learn meaningful patterns, since these only emerge clearly when there's enough data to separate them from noise.
- Few items dominate the high-rated category, which may bias recommendation systems and favor a TopPop approach.
- Most ratings are 5/5, likely because users tend to rate items they like more than those they dislike. This results in a highly imbalanced dataset for the task of predicting ratings.

2 Week 7 [Feb 10 - Feb 16]: Collaborative Filtering Recommender System

This week focused on implementing collaborative filtering recommender systems. The goal was to build models that predict user preferences based on past ratings, compare different approaches, and optimize their performance through hyperparameter tuning. These models serve as the foundation for generating personalized recommendations.

2.1 Top Popular (TopPop) Recommender System

As a baseline, I first implemented the **Top Popular (TopPop) recommender system**. This model recommends the most popular items based on the number of "high" ratings (ratings ≥ 3) in the training set. I computed the frequency of high ratings per item and selected the top-k items as recommendations for all users.

While computationally inexpensive, this method lacks personalization since it does not consider individual user preferences.

2.2 Collaborative Filtering Models

I then used the Scikit-Surprise library(Hug, 2020) to be able to compare TopPop with one neighborhood-based model and one latent factor model. The ones I chose are:

- **User-based k-Nearest Neighbors with Means (KNNWithMeans)**: A neighborhood-based approach that identifies similar users based on their rating history.
- **Singular Value Decomposition (SVD)**: A latent factor model that decomposes the user-item interaction matrix into lower-dimensional representations.

2.3 Hyperparameter Tuning and Evaluation

To optimize both models and finde the best hyperparameters, I performed a Grid Search with **5-fold cross-validation** on the training set. The parameters I tuned included:

- **User-based KNNWithMeans**
  - **k**: [5, 10, 20, 30, 40, 50, 70, 100]
  - **similarity_measure**: ['cosine', 'msd']
    * **cosine**: Computes the cosine of the angle between two vectors in a multidimensional space, ignoring magnitude and focusing on the direction of ratings.
    * **msd**: Computes the Mean Squared Difference between corresponding elements of two vectors, capturing the absolute differences in rating values.
- **SVD**
  - **n_factors**: [3, 5, 10, 20, 50, 100]
  - **n_epochs**: [5, 10, 20, 50, 100]

The evaluation metric chosen was **Root Mean Square Error (RMSE)**. RMSE penalizes larger errors more than smaller ones, making it a good measure to capture the overall predictive accuracy of our recommendation models. Lower RMSE indicates that the model predictions are closer to the actual ratings.

2.4 Results and Discussion

The best hyperparameters for each model can be found in Table 4.

Model	Best Hyperparameters	Avg RMSE
KNNWithMeans	'k': 70, 'similarity_measure': 'cosine'	0.9100
SVD	'n_factors': 5, 'n_epochs': 20	0.8440

Table 4: Best Hyperparameters and Average RMSE Across the 5 Folds for Each Model

The SVD model, despite it's low dimensional latent space, outperformed the KNNWithMeans model with a lower RMSE of 0.8440 compared to 0.9100. This indicates that the latent factor model is better at capturing the underlying patterns in the user-item interaction matrix.

After that, I ran the models with the optimal hyperparameters on the full training set and used them to predict the missing ratings. These predictions will later be compared to the actual ratings in the test set to assess model performance.

3 Week 8 [Feb 17 - Feb 23]: Evaluation of Recommender Systems

This week focused on evaluating the performance of the collaborative filtering models implemented in Week 7. The evaluation aimed to measure both the accuracy of rating predictions and the effectiveness of recommendations.

3.1 RMSE Evaluation

The first step in evaluating the models was measuring the **Root Mean Square Error (RMSE)** on the test set. RMSE quantifies how close the predicted ratings are to the actual ratings.

The trained models were tested on the preprocessed test data, and their RMSE values were computed and are shown in Table 5.

Model	Validation RMSE	Test RMSE
KNNWithMeans	0.9100	1.0619
SVD	0.8440	0.9795

Table 5: RMSE Comparison on Test Set

While RMSE is useful for evaluating rating prediction accuracy, it has some limitations. It does not consider ranking quality and does not directly reflect recommendation usefulness. For this reason, additional ranking-based metrics were computed.

3.2 Ranking-based Evaluation Metrics

To assess the quality of the generated recommendations, the models were evaluated on the test set using the following metrics:

- **Hit Rate**: Measures how often at least one relevant item appears in the top-k recommendations.
  - **Advantages:** Simple, easy to interpret.
  - **Disadvantages:** Doesn't consider ranking or multiple relevant items.
- **Precision@k**: Proportion of recommended items in the top-k list that are relevant.
  - **Advantages:** Direct measure of recommendation quality.
  - **Disadvantages:** Ignores total relevant items available and ranking order.
- **MAP@k**: Averages precision at positions of relevant items, rewarding good ranking.
  - **Advantages:** Considers ranking order.
  - **Disadvantages:** Sensitive to users with few relevant items.
- **MRR@k**: Focuses on the rank of the first relevant item. Computed as one divided by its rank.
  - **Advantages:** Rewards early relevant recommendations.
  - **Disadvantages:** Ignores additional relevant items.
- **Coverage**: Proportion of unique items recommended at least once.
  - **Advantages:** Indicates diversity in recommendations.
  - **Disadvantages:** High coverage doesn't mean better relevance.

The results of the computation of these metrics are shown in Table 6.

Model	Hit Rate	Precision@10	MAP@10	MRR@10	Coverage
KNNWithMeans	0.1092	0.0117	0.0084	0.0314	0.5874
SVD	0.0825	0.0083	0.0061	0.0222	0.1179
TopPop	0.2573	0.0325	0.0339	0.1187	0.0196

Table 6: Evaluation Metrics for Top-10 Recommendations on the Test Set (ratings ≥ 4 considered relevant)

3.3 Results and Discussion

- **Hit Rate:** Unsurprisingly, the **TopPop model** achieved the highest hit rate, likely because popular items frequently appear in the test set.
- **Precision@10:** The TopPop model outperformed the collaborative filtering methods in precision as well, suggesting that highly rated items tend to be good general recommendations.
- **MAP@10 and MRR@10:** TopPop also performed best in these ranking-based metrics, showing that the most popular items are often relevant.
- **Coverage:** The **kNN model** had the highest coverage (58.7%), indicating that it provided more diverse recommendations, while SVD had lower coverage. The TopPop model had the lowest coverage, meaning it recommended only a small subset of items.

The evaluation results show an interesting (but expected) trade-off:

- The **TopPop model** is simple but performs surprisingly well, especially in terms of hit rate and ranking metrics. However, it completely lacks of any sort of personalization.
- The **kNN model** achieves the highest coverage, meaning it suggests a wider variety of items, which could be useful for expanding user interest.
- The **SVD model** despite achieving the lowest RMSE, performed worse than expected in ranking-based evaluation, possibly due to the sparsity of the dataset.

3.4 What can be concluded

These results suggest that, while collaborative filtering methods are typically expected to perform well, the characteristics of the dataset (such as sparsity and rating distribution) play a significant role in determining effectiveness.

At this point, there is no clear winner, as the best model depends on the specific use case and the **trade-offs** between accuracy and coverage.

In a case where the only important thing is precision, **TopPop** is undoubtedly the best choice here. But the real hard yet rewarding thing to obtain is a wide coverage capable of recommending (even if with less confidence than TopPop) some niche items reviewed just by few users. In this regard, the higher coverage of **KNNWithMeans** proves to be very useful.

This approach also helps to spread the upcoming reviews, while **TopPop** would just end up making the popular items even more popular and the less common items even less common in proportion.

3.5 Analysis on the Effect of the Long-tail

To better understand how user and item activity impacts recommendation quality, I analyzed the performance of the models across different segments of the user and item distribution.

I divided users into top and bottom 20% based on rating count on the training set and computed Hit Rate on the test set for TopPop, KNNWithMeans, and SVD. The results are reported in Table 7.

Model	Top 20% Users	Bottom 20% Users
TopPop	0.164	0.371
KNNWithMeans	0.180	0.145
SVD	0.098	0.119

Table 7: Hit Rate for Top and Bottom 20% of Users

Interestingly, TopPop performs best for less active users. This aligns with its non-personalized nature: popular items are more likely to match the sparse preferences of cold-start users. On the other hand, KNN performs better for active users, benefiting from richer collaborative signals.

A similar analysis was performed for items, by computing on the test set the coverage for the top and bottom 20% most rated items of the train set. Table 8 shows the results.

Model	Top 20% Items	Bottom 20% Items
TopPop	0.099	0.000
KNNWithMeans	0.475	0.624
SVD	0.188	0.020

Table 8: Coverage for Top and Bottom 20% of Items

KNNWithMeans was the only model that consistently recommended long-tail items, which is crucial for discovering niche content. In contrast, TopPop completely ignores the long tail.

3.6 Error Analysis for the Neighborhood-based CF

To better understand the behavior of KNNWithMeans, I performed an error analysis based on Reciprocal Rank (RR). I selected one user with high RR (= 1.0) and one with low RR (= 0.0), and inspected their top-10 nearest neighbors.

I found that the average overlap in item history between the high-RR user and their neighbors was **1.30 items**, compared to only **1.00 item** for the low-RR user. This suggests that the model performs poorly when the user's rating history is dissimilar to their neighbors'.

Additionally, in cases with low RR, I noticed that many of the nearest neighbors had very different item preferences, which diluted the quality of the recommendations.

Conclusion: KNN models require sufficient user overlap. Sparse data leads to poor neighbor quality and weak predictions.

4 Week 9 [Feb 24 - Mar 2]: Text Representation

This week focused on Natural Language Processing (NLP) techniques to extract useful representations from the item descriptions. These textual features serve as the foundation for building content-based and hybrid recommendation models.

4.1 Preprocessing

To reduce noise and focus on semantic content, I cropped all item descriptions to the first 1000 characters, a requirement that was already satisfied by 84.48% of the items. Then, I applied the following preprocessing steps:

1. **Tokenization and lowercasing**.
2. **Punctuation removal**.
3. **Stopword removal**.
4. **Digit-containing token removal**.
5. **Stemming** (using the SnowballStemmer).

The vocabulary size was reduced from 51,296 to 27,243 tokens, decreasing sparsity and leading to denser representations.

4.2 Vector Representations

To represent the descriptions in vector spaces two main representations were used:

- **TF-IDF**: Sparse vectors built from the preprocessed descriptions. Final shape: $(\mathrm{n\ items}) \times$ 27,243.
- **Word2Vec**: Pretrained Google News embeddings were averaged over tokens to produce 300-dimensional dense vectors.

4.3 Cosine Similarity Analysis

I selected a subset of 10 random items and compared their pairwise similarities using each method. Figure 5 shows the heatmaps.

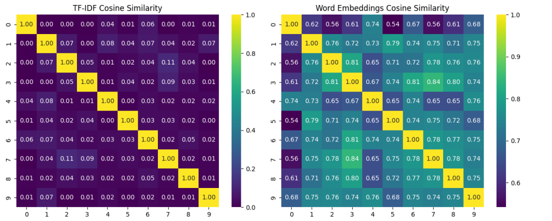

Figure 5: Cosine similarity between item vectors (TF-IDF vs Word2Vec)

TF-IDF focuses on exact term overlap, while Word2Vec captures broader semantic similarity.

Despite different mechanisms, both showed consistent patterns: low similarity for item 0 and high similarity between items 3 and 7, suggesting genuine content relationships.

Also, Word2Vec values were generally higher due to its ability to match conceptually related but lexically different terms, whereas TF-IDF produced a sparser similarity matrix.

4.4 Optional: BERT Embeddings

To explore contextualized embeddings, I used the ‘bert-base-uncased‘ model and extracted the 768-dimensional [CLS] token from the last hidden layer to represent each description. As with the other methods, I computed pairwise cosine similarities on the 10-item subset and visualized the results in Figure 6. The heatmap shows patterns similar to TF-IDF and Word2Vec but with notable differences, which suggests BERT picks up on different kinds of semantic relationships than the previous models.

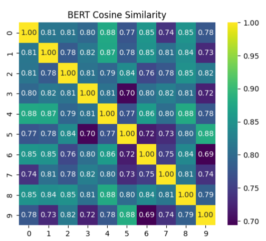

Figure 6: Cosine similarity between item vectors (BERT)

5 Week 10 [Mar 3 - Mar 9]: Content-Based Recommender System

In this phase, I leveraged the item representations obtained from textual data to build a content-based recommender system. The objective was to predict user preferences based solely on item content, without relying on collaborative signals.

5.1 Item and User Representation

Each **item** was encoded using TF-IDF vectors built from both the title and description fields. The preprocessing pipeline was the same as in Week 9, and the resulting vectors were concatenated into a single high-dimensional representation (41,550 features).

To represent each **user**, I averaged the vectors of the items they rated in the training set, creating a content profile that reflects their preferences. While the vectors weren't weighted by rating, the prevalence of 4 and 5-star ratings still skews the average toward preferred items.

5.2 Rating Prediction

To estimate a rating for a given user-item pair, I computed the cosine similarity between the user profile and the candidate item's TF-IDF vector. These similarity scores fall within the [0, 1] range. I then applied a simple linear transformation to map this value to the target [1, 5] rating range:

$$\text{predicted\_rating} = 1 + 4 \cdot \text{similarity}$$

Applied to 100 random ratings in the test set, this strategy yielded a high RMSE of **3.18**, mainly because most ratings are 5-stars, while cosine similarities tend to be low. Consequently, the linear mapping underestimates scores, failing to capture the skewed rating distribution.

5.3 Top-N Recommendation Strategy

Rather than relying on the predicted ratings directly, I used the cosine similarities themselves to produce ranked recommendation lists. For each user, all unrated items were sorted by their similarity to the user's profile vector, and the top-10 most similar items were selected as recommendations.

This similarity-based approach avoids distortion introduced by rescaling and directly reflects how well an item's textual content matches the user's historical preferences.

5.4 Evaluation

I evaluated on the test set the recommendations using the standard metrics. As before, items with test ratings ≥ 4 were considered relevant.

Model	Hit Rate	Precision@10	MAP@10	MRR@10	Coverage
Content-Based	0.0121	0.0012	0.0007	0.0022	0.0485

Table 9: Content-Based Recommender Evaluation Results

The results show limited predictive power, likely due to the semantic gap between text-based similarity and actual user preferences. Still, the model has decent item diversity and is particularly useful for cold-start items.

6 Week 11 [Mar 10 - Mar 16]: Hybrid Recommender Systems

This week focused on combining collaborative filtering and content-based approaches to build hybrid recommenders. The goal was to achieve better personalization, robustness, and item coverage.

6.1 Implemented Hybrid Strategies

I experimented with three hybridization techniques:

1. Parallel Hybrid The collaborative (KNNWithMeans) and content-based models independently ranked all items. Each item's final score was computed as the sum of $k - \text{rank}(i)$ from both models, and the top-k items were selected accordingly.

$$\text{score}(i) = (k - \text{rank}_{\text{CF}}(i)) + (k - \text{rank}_{\text{CB}}(i))$$

2. Switching Hybrid Users with more than 5 interactions used collaborative filtering, while users with fewer interactions were assigned content-based recommendations. This method adapts to the user's data availability.

3. Pipeline Hybrid For each user, the top-25 items most similar to their profile (from the content-based model) were selected as candidates. These were re-ranked using KNNWithMeans scores.

6.2 Evaluation and Comparison

All models were evaluated on the test set using the same metrics as before. The results are shown in Table 10.

Model	Hit Rate	Precision@10	MAP@10	MRR@10	Coverage
TopPop Baseline	0.2573	0.0325	0.0339	0.1187	0.0196
KNN Collaborative	0.1092	0.0117	0.0084	0.0314	0.5874
SVD Collaborative	0.0825	0.0083	0.0061	0.0222	0.1179
Switching Hybrid	0.0825	0.0083	0.0077	0.0262	0.0526
Parallel Hybrid	0.0583	0.0058	0.0048	0.0186	0.0944
Pipeline Hybrid	0.0194	0.0019	0.0031	0.0134	0.0486
Content-Based	0.0121	0.0012	0.0007	0.0022	0.0485

Table 10: Comparison of selected recommenders on the test set (ratings ≥ 4 considered relevant)

Among the three hybrid systems, the **Switching Hybrid** performed best across most metrics. It capitalized on the strengths of CF for well-known users while handling cold-starts with content-based fallback. The **Parallel Hybrid** had the highest coverage, benefiting from both sources of diversity. The **Pipeline Hybrid**, although conceptually appealing, performed worse, possibly due to candidate filtering removing relevant items.

6.3 Visual Comparison with Star Plot

To visually compare the models, I created a star plot (Figure 7) using the five evaluation metrics on the test set. Each axis represents one metric (normalized), and the area enclosed reflects overall performance.

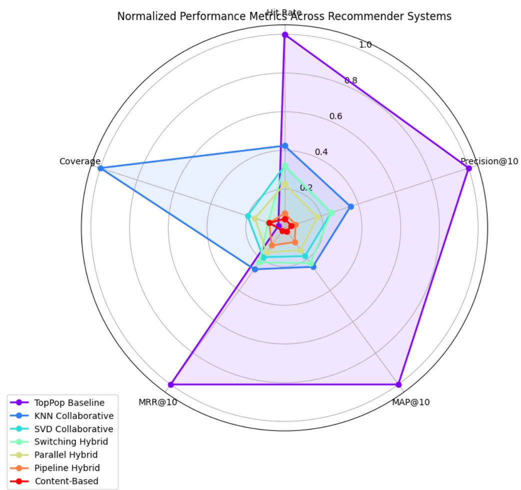

Figure 7: Star plot comparing performances

This visualization highlights key trade-offs among the models. TopPop achieves strong performance across most metrics but suffers from minimal coverage, as it only recommends a small set of popular items. KNN-based collaborative filtering provides the best balance between accuracy and diversity but struggles with cold-start items, making it difficult to recommend new products without prior interactions. Hybrid approaches, though their raw metrics may seem modest, integrate content signals to enhance robustness, address sparsity, and enable recommendations for unseen items.

References

Nicolas Hug. 2020. Surprise: A python library for recommender systems. *Journal of Open Source Software*, 5(52):2174.